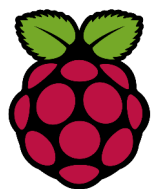


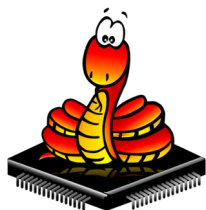


Atelier Raspi N°21

La programmation
Le code Morse

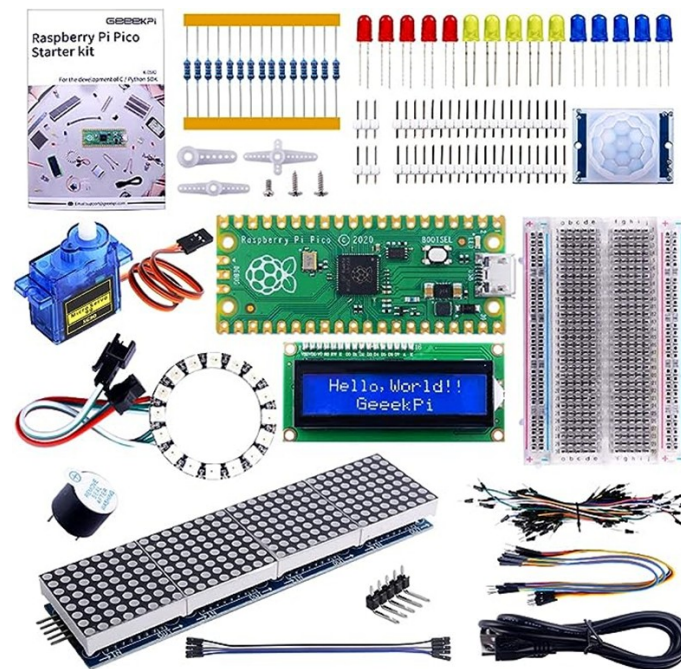


Logo du Raspberry Pico



Logo du MicroPython

L'atelier a pour valeurs, le partage, l'aide, la formation, le faire et construire ensemble à partir de l'expérience des participants



- **La programmation**

- Etape 1 Programmation – Langage MicroPython

- Etape 2 Les variables dans Python

- Etape 3 Les types de variables et opérateurs standards

- Etape 4 Les structures de contrôle 1

- Etape 5 Les structures de contrôle 2

- Etape 6 Les structures de contrôle 3

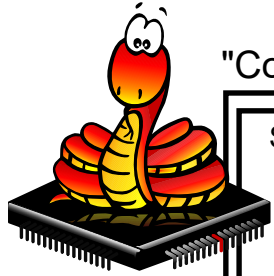
- Etape 7 Listes et dictionnaires

- Etape 8 Fonctions

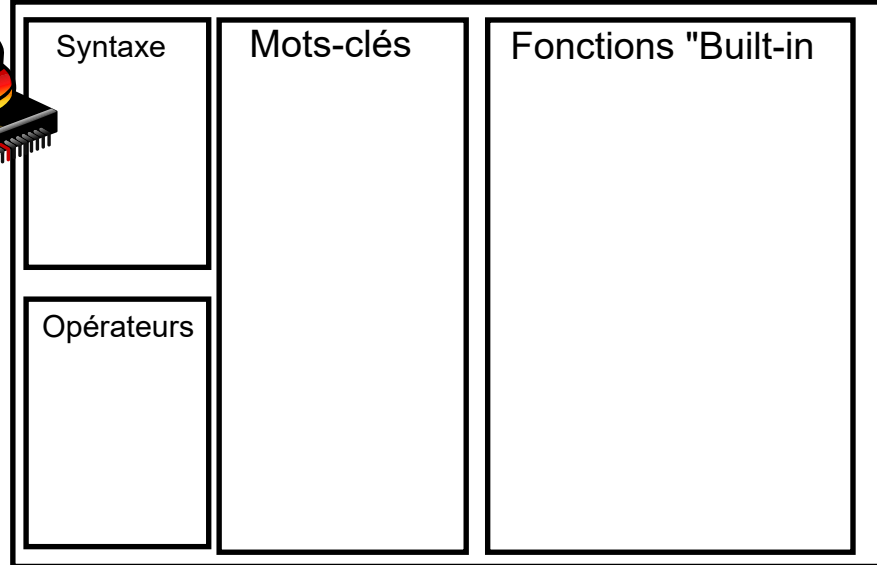
- Etape 9 Fonctions natives, fonction intégrée à python

- Etape 10 Modules

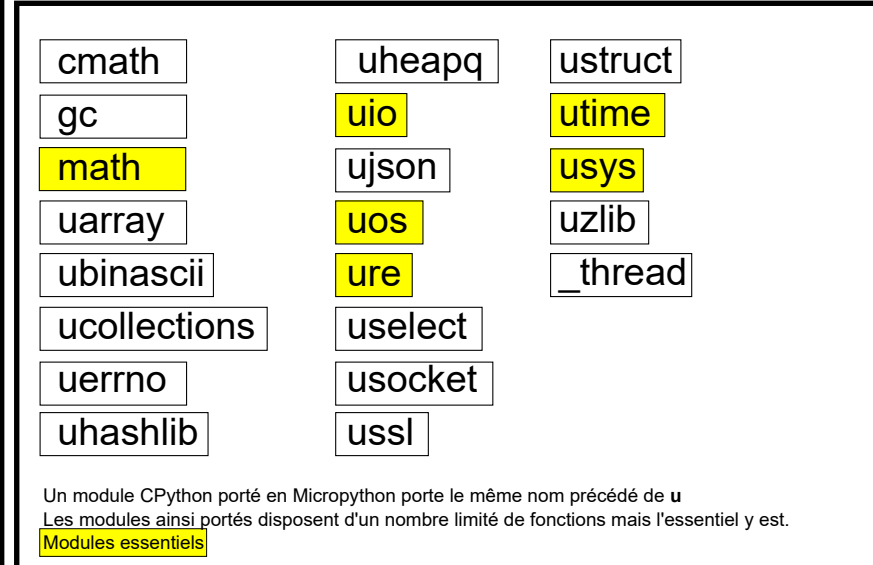
A21E1 Programmation – Langage MicroPython



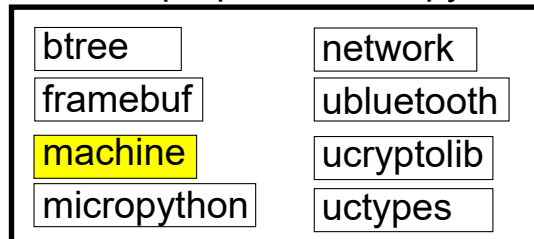
"Core" Micropython



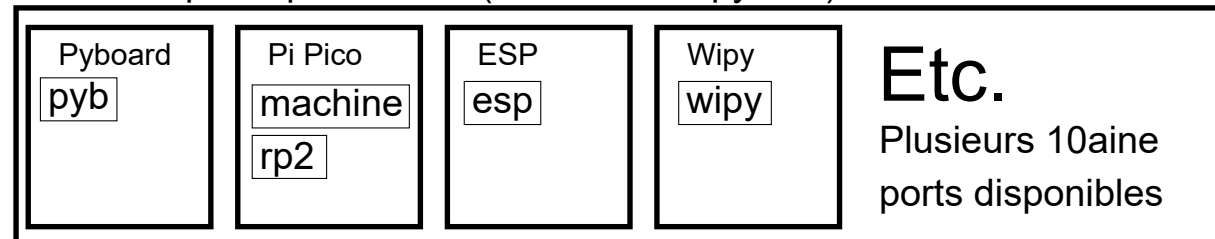
Modules standards



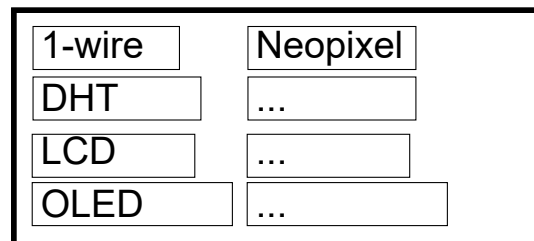
Modules propres à Micropython



Modules spécifiques cartes ("Ports" Micropython)



Modules "librairies" hardware



A21E2 Les variables

Une variable est un espace mémoire dans lequel on va stocker des objets.

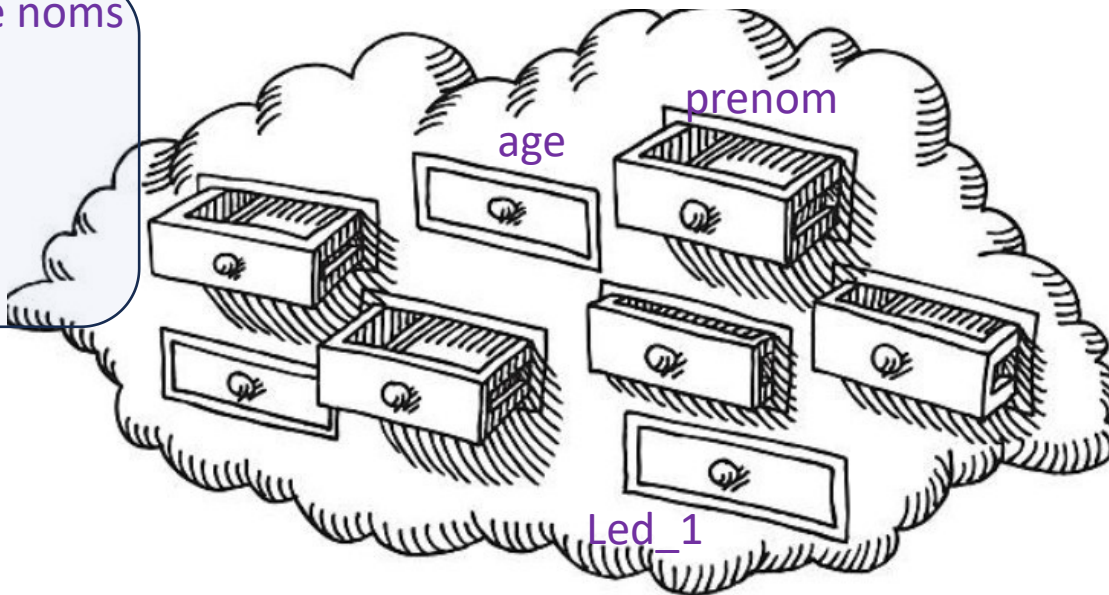
C'est comme un tiroir où l'on peut ranger (écrire), regarder (lire) ou enlever (supprimer) un ou plusieurs objets. On va donner un nom à cette variable afin de pouvoir l'identifier, comme on met une étiquette sur un tiroir.

Ce nom doit suivre quelques règles :

- être compréhensible : décrire à quoi sert ce tiroir
- commencer par un caractère et non par un chiffre
- ne pas contenir de caractère spécial ni accentué
- ne pas contenir d'espace, si l'on souhaite donner un nom composé, utiliser le caractère souligné (underscore) '_'
- il faut rester cohérent dans la façon de nommer les variables

Exemples de noms de variable:

age
led_1
led_2



Affectation:

Pour écrire un objet dans une variable, on utilise l'opérateur '=' en mettant le nom de la variable à gauche de l'opérateur et l'objet à associé à droite.

Exemple:

age = 10

On peut lire le contenu de notre variable pour l'afficher:

print(age)

A21E3 Les types de variables et opérateurs standards

Une variable sans valeur (tiroir vide) est définie par : *age = None*

Type (objet) de base:

- **bool** : définit un booléen qui peut prendre les valeurs **True** ou **False** (*Vrai ou faux*)
- **int** : définit un nombre entier (ex: 10)
- **float** : définit un nombre réel (ex :10,2)
- **str** : définit une chaîne de caractère (string, ex : « Bonjour tout le monde ! »)

Lors d'un test les valeurs suivantes sont considérée comme fausse : **False, None, "", [], (), {}**

Opération mathématique:

- **+** : addition, ex: **10+3** -> 13
- **-** : sustraction, ex: **10-3** -> 7
- ***** : multiplication, ex: **5*2** -> 10
- **/** : division, ex: **20/2** -> 10
- ****** : élévation a la puissance, ex: **6**4** -> 1296
- **//** : division entière, ex: **6//4** -> 1
- **%** : reste de la division entière, ex: **6%4** -> 2

Opérateurs de comparaison:

- **==** : égalité, **None==None** est vrai
- **!=** : inégalité, **None!=None** est faux
- **>, >=, <, <=** : comparaison, **6>3** est vrai
- **is, is not** : permet de comparer l'identité des objets

Opérateurs booléens:

- **and** : **True and False est faux**
- **or** : **True and False est vrai**
- **not** : **not False** est vrai

A21E4 Les structures de contrôle- 1

Indentation

En python l'indentation est obligatoire et permet de définir un bloc d'une ou plusieurs instructions, Une indentation est un retrait du code de 4 caractères, elle peut être insérée par 4 'espace' ou par la touche tabulation 'TAB'

Le symbole ':' à la fin d'une ligne d'instruction marque le début d'un bloc.

Seules quelques instructions permettent de définir le début d'un bloc:

- **def** : définition d'une fonction
- **if** : début d'un test
- **while et for** : début d'une boucle

A21E5 Les structures de contrôle- 2

Comparaison

Sert à comparer deux valeurs, le résultat d'une comparaison est un booléen qui prend la valeur **True** ou **False**.

< : strictement inférieur

<= : inférieur ou égal

== : égal

!= : différent

> : strictement supérieur

>= : supérieur ou égal

Exemple : **3 > 2 -> True**

L'instruction if, if...else, if...elif...else

L'instruction **if** Permet d'exécuter un bloc si la condition est vraie (**True**). Si ce n'est pas le cas on peut continuer le programme sans traiter le bloc d'instruction ou décider de traiter un autre bloc d'instruction délimité soit par **elif** si on souhaite rendre ce nouveau bloc conditionnel, soit par **else** si on souhaite le traiter par défaut.

Exemple:

n = -3

print(f 'le nombre {n} est: ')

if n > 0:

print('positif')

elif n < 0:

print('négatif')

else :

print('zéro')

A21E6 Les structures de contrôle- 3

Boucle while

La boucle **while** exécute un bloc tant que la condition est vraie (**True**).

Exemple:

```
n = 3  
while n > 0:  
    print(n)  
    n = n - 1  
print('fini !')
```

Boucle for

La boucle **for** permet de parcourir une liste d'objet une à une dans l'ordre de la première à la dernière.

Exemple:

```
for c in 'hello':
```

```
    print(c)
```

la variable **c** va prendre tour à tour les valeurs 'h', 'e', 'l', 'l', 'o'

A21E7 Listes et dictionnaires

Listes

La liste est une séquence linéaire d'objet. Elle est délimitée par des **[]** ou **des ()** et ses éléments sont séparés par des **' , '**. Une liste peut contenir plusieurs types d'objet. On peut utiliser le constructeur **list()** pour créer une liste vide,

Exemple:

N = [123, 25, 12]

B = ['chat', 12, 'chien']

La fonction **len** permet de connaître la longueur d'une liste (son nombre d'élément):

len(N) retourne la valeur 3

On peut parcourir une liste par itération:

for item in N:

print(item)

Ou par index:

for index in range(len(N)):

print(N[index])

Dictionnaires

Comme les listes les dictionnaires peuvent contenir plusieurs objets de type différents. Ceux-ci seront toujours sous la forme d'une paire **key:valeur**. Le constructeur **dict()** permet de créer un dictionnaire vide. Un dictionnaire est délimité par **{ }**.

Exemple:

N = {'voiture':'4 roues', 'moto':'2 roues'}

B = {'animal':'chat', 'taille_poids':[60,10]}

On accède à la valeur d'un **key** en l'utilisant comme index:

print(N[voiture]) affiche : '4 roues'

A21E8 Fonctions

Fonctions

Une fonction permet d'écrire un bloc de code que l'on va pouvoir appeler plusieurs fois à différent moment de notre programme sans avoir à le réécrire. Une fonction doit être définie avant de pouvoir être appelée. On définit une fonction à l'aide du mot clé ***def nom_de_ma_fonction(args)*** : on peut ensuite l'appeler par ***nom_de_ma_fonction()*** depuis notre programme principal ou depuis une autre fonction.

Une fonction peut prendre un ou plusieurs arguments en entrée et retourner un objet.

Exemple:

```
def saluer(nom):  
    print(f'bonjour {nom}')
```

Utilisation de la fonction:

```
saluer('Philippe')
```

```
Saluer('marc')
```

Rendu du programme:

```
Bonjour Philippe
```

```
Bonjour marc
```

A21E9 Fonctions natives, fonction intégrée à python

• abs • aiter • all • anext • any • ascii • bin • bool • breakpoint • bytearray • bytes • callable • chr • classmethod • compile • complex • delattr • dict • dir • divmod • enumerate • eval • exec • filter • float • format • frozenset • getattr • globals • hasattr • hash • help • hex	• Retourne la valeur absolue d'un nombre • Renvoie un itérateur asynchrone • Retourne True si tous les éléments d'un itérable sont vrais • Renvoie l'élément suivant d'un itérable asynchrone • Retourne True si au moins un élément d'un itérable est vrai • Retourne une string qui représente l'objet • Converti un nombre en binaire sous forme de string • Converti en élément en valeur booléenne • Place dans le débogueur • Retourne un tableau de taille donnée • Retourne un objet de type "byte" • Test si l'objet est "appelable" • Converti un nombre en sa valeur ascii sous forme de string • Retourne la méthode de classe pour une fonction • Exécute puis retourne du code Python • Retourne un nombre complexe • Supprime un attribut de l'objet • Crée et retourne un dictionnaire {} • Retourne une liste des attributs d'un objet Python • Retourne un tuple avec le quotient et le reste d'une division • Retourne un objet enumerate • Exécute du code Python • Exécute du code dynamiquement • Crée un itérateur à partir d'éléments qui renvoient True • Retourne un nombre à virgule flottante • Retourne une représentation formatée d'une valeur • Retourne un objet immutable frozenset • Retourne la valeur de l'attribut nommé d'un objet • Retourne un dictionnaire avec la table des symboles globaux • Test si la l'objet a l'attribut ou pas • Retourne un entier avec la valeur de hachage d'un objet • Appelle l'aide native de Python • Converti un entier en nombre de base hexadécimale
--	--

• id • input • int • isinstance • isinstance • iter • len • list • locals • map • max • memoryview • min • next • object • oct • open • ord • pow • print • property • range • repr • reversed • round • set • setattr • slice • sorted • staticmethod • str • sum • super • tuple • type • vars • zip • import	• Renvoie l'identification d'un objet • Lit et renvoie une ligne de chaînes de caractères • Converti un nombre ou une string en nombre entier • Test si un objet est une instance de la classe • Test si un objet est une instance d'une autre classe • Retourne un itérateur • Retourne la longueur d'un objet • Crée une nouvelle liste ou transforme un autre objet en liste • Retourne un dictionnaire avec la table des symboles locaux • Applique une fonction et retourne une liste • Retourne le plus grand élément • Retourne une identification mémoire d'un objet de type byte • Retourne le plus petit élément • Retourne l'élément suivant d'un itérateur • Crée un objet Python • Converti un entier en nombre de base 8 • Retourne un objet de type fichier • Retourne un entier d'un caractère avec sa valeur unicode • Retourne la puissance d'un nombre • Permet d'afficher un objet sur la sortie standard • Retourne un objet "property" • Retourne une liste de nombres • Retourne une représentation d'un objet • Retourne un itérateur dans le sens inverse • Retourne un nombre arrondi à une décimale donnée • Crée et retourne un set d'éléments uniques (sans doublon) • Définit la valeur d'un attribut de l'objet • Coupe et retourne un objet • Trie et retourne un itérable • Transforme une méthode en une méthode statique • Retourne une chaîne de caractères • Fait la somme des éléments d'un itérable • Retourne un objet proxy de la classe • Crée et retourne un tuple • Retourne le type d'un objet Python • Retourne l'attribut __dict__ • Prend des itérables et retourne une liste de tuples • Fonction appelé par l'instruction import
---	--

A21E10 Modules

Modules

Un module contient un ou plusieurs fichier python que l'on peut importer en début de script et qui contient un ensemble de définitions. Exemple le module ***time*** contient des fonctions permettant de dormir.

Exemple:

```
import time
```

```
time.sleep(3) # dort pendant 3 secondes
```

Un module spécifique à l'utilisation du raspi pico existe, c'est lui qui va nous permettre d'interagir avec les périphériques du microcontrôleur et donc avec le monde réel : ***machine***

Exemple:

```
from machine import Pin → depuis le module machine on déclare que l'on va utiliser l'objet Pin
```

```
P0 = Pin(0, Pin.OUT) → On déclare une variable P0 dans laquelle on range un abjet Pin particulier : la broche 0 du pico et on définit que ce sera une sortie
```

```
P0.value(1) → On peut maintenant utiliser les fonctions propre à l'objet Pin, ici la fonction (méthode) value auquel on passe un argument (paramètre) '1' signifiant que l'on souhaite que la broche prenne un état électrique haut (vcc)
```

- **Application : Le code Morse**
 - Présentation du code Morse
 - Câblage des 2 LEDs
 - Le code

LE CODE MORSE

Un langage simple, des messages sans fil !

Le code Morse est un système de communication qui utilise des signaux courts et longs pour représenter des lettres, des chiffres et des signes de ponctuation.

Simple à comprendre, difficile à oublier !



Comment ça marche ?

● = un point (court)
durée : 1 unité de temps

— = un trait (long)
durée : 3 unités de temps

Entre les éléments d'une même lettre : 1 unité

Entre les lettres : 3 unités

Entre les mots : 7 unités



Lettres de l'alphabet

Lettre	Code	Lettre	Code
A	● —	N	— ●
B	— ● ● ●	O	— — —
C	— ● — ●	P	● — — ●
D	— ● ●	Q	— — ● —
E	●	R	● — ●
F	● ● — ●	S	● ● ●
G	— — ●	T	—
H	● ● ● ●	U	● ● —
I	● ●	V	● ● ● —
J	● — — —	W	● — —
K	— ● —	X	— ● ● —
L	● — ● ●	Y	— ● — —
M	— —	Z	— — ● ●

Chiffres

Chiffre	Code
1	● — — — —
2	● ● — — —
3	● ● ● — —
4	● ● ● ● —
5	● ● ● ● ●
6	— ● ● ● ●
7	— — ● ● ●
8	— — — ● ●
9	— — — — ●
0	— — — — —

Quelques signes de ponctuation

Signe	Code
.	● — ● — ● —
,	— — ● ● — —
?	● ● — — ● ●
/	— ● ● — ●
(	— ● — — ●
)	— ● — — —
:	— — — ● ● ●
;	— ● — — ● ●
=	— ● ● ● —
'	● — — — — ●
@	● — — ● — ●

Exemple de message

Déchiffrer : SOS (à l'aide du tableau)

● ● ● — — — ● ● ●
S O S

Coder : HELLO (à toi de jouer !)

● ● ● ● — — ● ● ● — — — ● ● ●
H E L L O



Astuce : commence par écrire les lettres dans ton cahier pour bien les mémoriser !

Le matériel (version simple)



Une clé, un buzzer ou une LED
(avec un microcontrôleur comme un Raspberry Pi Pico)
et c'est parti !

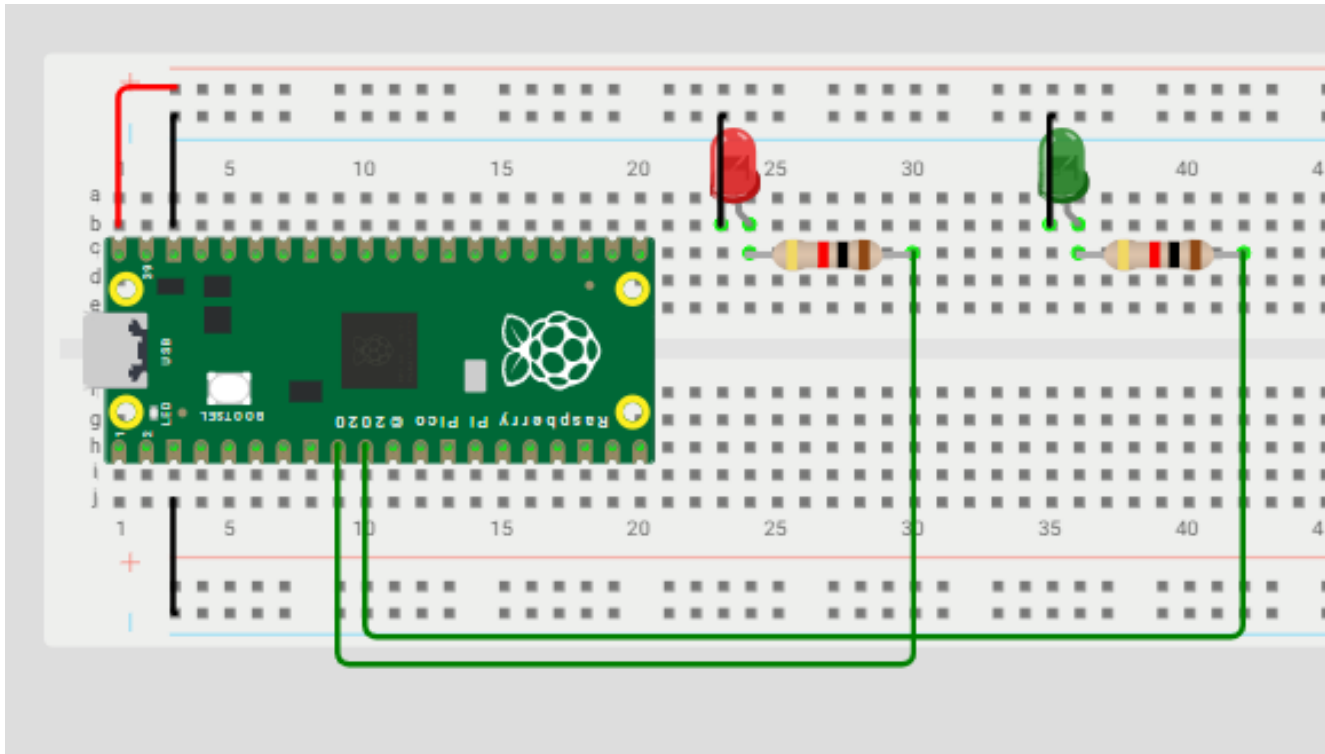
À retenir

- ✓ 1 point = 1 unité | 1 trait = 3 unités
- ✓ 3 unités entre les lettres | 7 unités entre les mots
- ✓ 26 lettres + 10 chiffres + ponctuation
- ✓ Un code simple, efficace et toujours utilisé aujourd'hui !

Le Morse,
c'est aussi
une histoire
d'inventivité !

Câblage des 2 LEDs

- * Faire vérifier le montage avant connexion du microcontrôleur au PC
- * Ne pas mettre les doigts sur le circuit imprimé du microcontrôleur
- * Tension max sur chaque Pin = 3,3V en Entrée comme en Sortie



Composant	GPIO
Led 1 (rouge)	6
Led 2 (verte)	7

Code couleur des câbles :

- Vcc : Rouge / Orange
- Gnd : Noir / Marron
- Données : Vert / Bleu / Jaune / Violet

Le programme

```
3 from machine import Pin
4 import time
5 time.sleep(0.1) # Wait for USB to become ready
6
7 print("Hello, Pi Pico!")
8
9 led_lettre = Pin(25, Pin.OUT)
10 led_morse = Pin(9, Pin.OUT)
11
12 TIRET_TIME = 1
13 POINT_TIME = TIRET_TIME/2
14
15 S = (".", ".", ".")
16 O = ("-", "-", "-")
17
18 message = (S,O,S)
19
20
21 led_lettre.value(0)
22 led_morse.value(0)
23
24 print ("debut du message")
25
26 for lettre in message:
27     for symbole in lettre:
28         led_morse.value(1)
29         if symbole == "-":
30             time.sleep(TIRET_TIME)
31         else:
32             time.sleep(POINT_TIME)
33         led_morse.value(0)
34         time.sleep(POINT_TIME)
35
36     led_lettre.toggle()
37
38
39 print("fin du message")
```